JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Boolean

java.lang.Object
    java.lang.Boolean

All Implemented Interfaces:

Serializable, Comparable<Boolean>, Constable

```
public final class Boolean
extends Object
implements Serializable, Comparable<Boolean>, Constable
```

The Boolean class is the wrapper class for values of the primitive type boolean. An object of type Boolean contains a single field whose type is boolean.

In addition, this class provides many methods for converting a boolean to a String and a String to a boolean, as well as other constants and methods useful when dealing with a boolean.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail.

Since:

1.0

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static final Boolean	FALSE	The Boolean object corresponding to the primitive value false.
static final Boolean	TRUE	The Boolean object corresponding to the primitive value true.
static final Class<Boolean>	TYPE	The Class object representing the primitive type boolean.

Constructor Summary

Constructors

Constructor	Description
`Boolean(boolean value)`	**Deprecated.** It is rarely appropriate to use this constructor.
`Boolean(String s)`	**Deprecated.** It is rarely appropriate to use this constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`boolean`	`booleanValue()`	Returns the value of this `Boolean` object as a boolean primitive.
`static int`	`compare(boolean x, boolean y)`	Compares two `boolean` values.
`int`	`compareTo(Boolean b)`	Compares this `Boolean` instance with another.
`Optional<DynamicConstantDesc·`	`describeConstable()`	Returns an `Optional` containing the nominal descriptor for this instance.
`boolean`	`equals(Object obj)`	Returns `true` if and only if the argument is not `null` and is a `Boolean` object that represents the same `boolean` value as this object.
`static boolean`	`getBoolean(String name)`	Returns `true` if and only if the system property named by the argument exists and is equal to,

		ignoring case, the string `"true"`.
`int`	**`hashCode`**`()`	Returns a hash code for this `Boolean` object.
`static int`	**`hashCode`**`(boolean value)`	Returns a hash code for a `boolean` value; compatible with `Boolean.hashCode()`.
`static boolean`	**`logicalAnd`**`(boolean a, boolean b)`	Returns the result of applying the logical AND operator to the specified `boolean` operands.
`static boolean`	**`logicalOr`**`(boolean a, boolean b)`	Returns the result of applying the logical OR operator to the specified `boolean` operands.
`static boolean`	**`logicalXor`**`(boolean a, boolean b)`	Returns the result of applying the logical XOR operator to the specified `boolean` operands.
`static boolean`	**`parseBoolean`**`(String s)`	Parses the string argument as a boolean.
`String`	**`toString`**`()`	Returns a `String` object representing this Boolean's value.
`static String`	**`toString`**`(boolean b)`	Returns a `String` object representing the specified boolean.
`static Boolean`	**`valueOf`**`(boolean b)`	Returns a `Boolean` instance representing the specified `boolean` value.
`static Boolean`	**`valueOf`**`(String s)`	

	Returns a `Boolean` with a value represented by the specified string.

Methods declared in class Object

`clone`, `finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Field Details

TRUE

```
public static final Boolean TRUE
```

The `Boolean` object corresponding to the primitive value `true`.

FALSE

```
public static final Boolean FALSE
```

The `Boolean` object corresponding to the primitive value `false`.

TYPE

```
public static final Class<Boolean> TYPE
```

The Class object representing the primitive type boolean.

Since:

1.1

Constructor Details

Boolean

```
@Deprecated(since="9")
public Boolean(boolean value)
```

Deprecated.
It is rarely appropriate to use this constructor. The static factory `valueOf(boolean)` is generally a better choice, as it is likely to yield significantly better space and time performance. Also consider using the final fields `TRUE` and `FALSE` if possible.

Allocates a `Boolean` object representing the `value` argument.

Parameters:

value - the value of the Boolean.

Boolean

```
@Deprecated(since="9")
public Boolean(String s)
```

> **Deprecated.**
> *It is rarely appropriate to use this constructor. Use `parseBoolean(String)` to convert a string to a `boolean` primitive, or use `valueOf(String)` to convert a string to a `Boolean` object.*

Allocates a Boolean object representing the value true if the string argument is not null and is equal, ignoring case, to the string "true". Otherwise, allocates a Boolean object representing the value false.

Parameters:

s - the string to be converted to a Boolean.

Method Details

parseBoolean

```
public static boolean parseBoolean(String s)
```

Parses the string argument as a boolean. The boolean returned represents the value true if the string argument is not null and is equal, ignoring case, to the string "true". Otherwise, a false value is returned, including for a null argument.

Example: Boolean.parseBoolean("True") returns true.
Example: Boolean.parseBoolean("yes") returns false.

Parameters:

s - the String containing the boolean representation to be parsed

Returns:

the boolean represented by the string argument

Since:

1.5

booleanValue

```
public boolean booleanValue()
```

Returns the value of this Boolean object as a boolean primitive.

Returns:

the primitive `boolean` value of this object.

valueOf

```
public static Boolean valueOf(boolean b)
```

Returns a `Boolean` instance representing the specified `boolean` value. If the specified `boolean` value is `true`, this method returns `Boolean.TRUE`; if it is `false`, this method returns `Boolean.FALSE`. If a new `Boolean` instance is not required, this method should generally be used in preference to the constructor `Boolean(boolean)`, as this method is likely to yield significantly better space and time performance.

Parameters:

`b` - a boolean value.

Returns:

a `Boolean` instance representing `b`.

Since:

1.4

valueOf

```
public static Boolean valueOf(String s)
```

Returns a `Boolean` with a value represented by the specified string. The `Boolean` returned represents a true value if the string argument is not `null` and is equal, ignoring case, to the string `"true"`. Otherwise, a false value is returned, including for a null argument.

Parameters:

`s` - a string.

Returns:

the `Boolean` value represented by the string.

toString

```
public static String toString(boolean b)
```

Returns a `String` object representing the specified boolean. If the specified boolean is `true`, then the string `"true"` will be returned, otherwise the string `"false"` will be returned.

Parameters:

`b` - the boolean to be converted

Returns:

the string representation of the specified `boolean`

Since:

1.4

toString

```
public String toString()
```

Returns a `String` object representing this Boolean's value. If this object represents the value `true`, a string equal to `"true"` is returned. Otherwise, a string equal to `"false"` is returned.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this object.

hashCode

```
public int hashCode()
```

Returns a hash code for this `Boolean` object.

Overrides:

`hashCode` in class `Object`

Returns:

the integer `1231` if this object represents `true`; returns the integer `1237` if this object represents `false`.

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

hashCode

```
public static int hashCode(boolean value)
```

Returns a hash code for a `boolean` value; compatible with `Boolean.hashCode()`.

Parameters:

`value` - the value to hash

Returns:

a hash code value for a `boolean` value.

Since:

1.8

equals

`public boolean equals(Object obj)`

Returns `true` if and only if the argument is not `null` and is a `Boolean` object that represents the same `boolean` value as this object.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to compare with.

Returns:

`true` if the Boolean objects represent the same value; `false` otherwise.

See Also:

`Object.hashCode()`, `HashMap`

getBoolean

`public static boolean getBoolean(String name)`

Returns `true` if and only if the system property named by the argument exists and is equal to, ignoring case, the string `"true"`. A system property is accessible through `getProperty`, a method defined by the `System` class.

If there is no property with the specified name, or if the specified name is empty or null, then `false` is returned.

Parameters:

`name` - the system property name.

Returns:

the `boolean` value of the system property.

See Also:

`System.getProperty(java.lang.String)`,
`System.getProperty(java.lang.String, java.lang.String)`

compareTo

`public int compareTo(Boolean b)`

Compares this `Boolean` instance with another.

Specified by:

`compareTo` in interface `Comparable<Boolean>`

Parameters:

`b` - the `Boolean` instance to be compared

Returns:

zero if this object represents the same boolean value as the argument; a positive value if this object represents true and the argument represents false; and a negative value if this object represents false and the argument represents true

Throws:

`NullPointerException` - if the argument is `null`

Since:

1.5

See Also:

`Comparable`

compare

```
public static int compare(boolean x,
                          boolean y)
```

Compares two `boolean` values. The value returned is identical to what would be returned by:

```
   Boolean.valueOf(x).compareTo(Boolean.valueOf(y))
```

Parameters:

`x` - the first `boolean` to compare

`y` - the second `boolean` to compare

Returns:

the value `0` if `x == y`; a value less than `0` if `!x && y`; and a value greater than `0` if `x && !y`

Since:

1.7

logicalAnd

```
public static boolean logicalAnd(boolean a,
                                 boolean b)
```

Returns the result of applying the logical AND operator to the specified `boolean` operands.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the logical AND of `a` and `b`

Since:

1.8

See Also:

`BinaryOperator`

logicalOr

```
public static boolean logicalOr(boolean a,
                                boolean b)
```

Returns the result of applying the logical OR operator to the specified `boolean` operands.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the logical OR of `a` and `b`

Since:

1.8

See Also:

`BinaryOperator`

logicalXor

```
public static boolean logicalXor(boolean a,
                                 boolean b)
```

Returns the result of applying the logical XOR operator to the specified `boolean` operands.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the logical XOR of `a` and `b`

Since:

1.8

See Also:

`BinaryOperator`

describeConstable

```
public Optional<DynamicConstantDesc<Boolean>> describeConstable()
```

Returns an `Optional` containing the nominal descriptor for this instance.

Specified by:

`describeConstable` in interface `Constable`

Returns:

an `Optional` describing the `Boolean` instance

Since:

15

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.